TRINITY COLLEGE · OCTOBER 5, 2026

AI Integration

Retrieval-augmented generation, and the plan

Ken Kousen · CPSC 415
Monday 1:30–4:10 · MC-225

Where we are

Last week

Your program asked for data
Five cases said whether it worked
You wrote the spec before code

This week

Your program answers from documents
Five questions, with and without them
You approve a plan before code
The key moves to a server

> A spec, reviewed and corrected: kousen/cpsc415-week02, commit "Approve spec".

REST in one page

Verb	Means	Repeat safely?
`GET`	read	yes
`POST`	create, or run an action	no
`PUT` / `PATCH`	replace / change part	`PUT` yes
`DELETE`	remove	yes

- A URL names a thing; the verb says what to do.
- **Stateless:** every request carries the key and the whole conversation.
- AI APIs are almost all `POST`: every call is a computation you pay for.

Handout: weeks/04/rest.md

The problem RAG solves

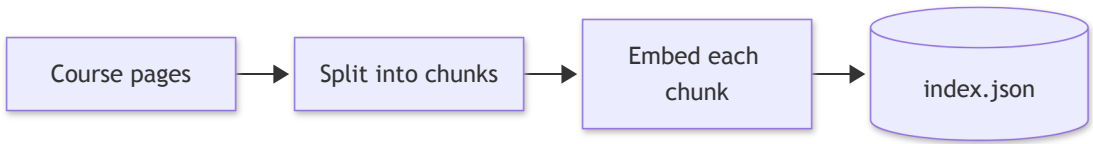
- The model knows its training data and whatever is in the prompt. Nothing else.
- Your documents are private, new, or too long to paste in every time.
- So at question time, send only **the few pieces that matter**.

> Retrieve the relevant chunks. Put them in the prompt. Generate the answer.

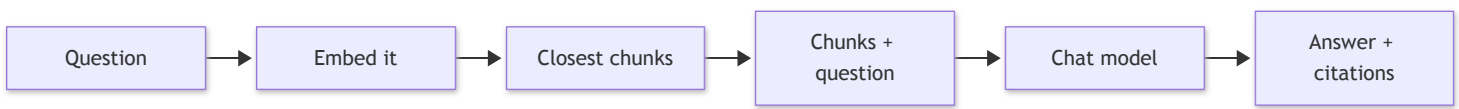
Video: youtu.be/3kMwv7Kraxk · code: github.com/kousen/ragdemo

Two phases

Once, and again whenever the pages change



For each question



Frameworks call a chunk a segment, and the index an embedding store or vector store.

What an embedding is

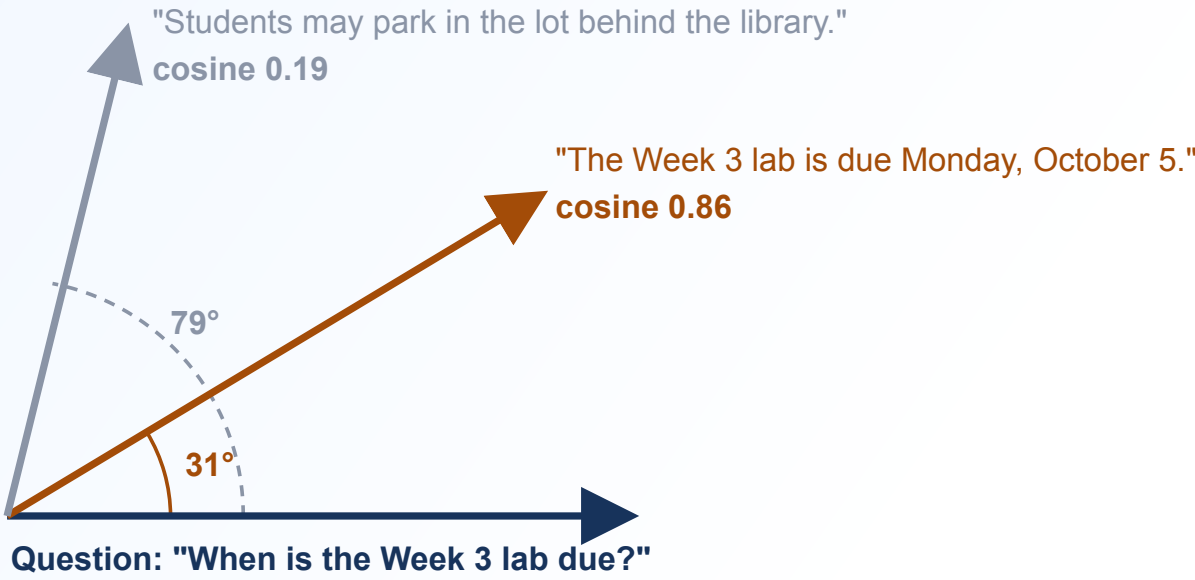
- A list of numbers for a piece of text: 1,536 of them for `text-embedding-3-small`.
- Texts that mean similar things point in similar directions.
- **Cosine similarity** measures the angle: 1.0 is the same direction.

```
def norm(v):
    return math.sqrt(sum(x * x for x in v))

def cosine(a, b):
    return sum(x * y for x, y in zip(a, b)) / (norm(a) * norm(b))
```

Some tools report similarity (higher is closer), some report distance (lower is closer). Same idea.

Three sentences, three directions



> Rank every chunk by cosine against the question; the top k go into the prompt.

Measured with text-embedding-3-small on Oct 5, 2026, and drawn to scale in 2D. The real vectors have 1,536 dimensions; the angle math is identical.

Live: index the course

```
python3 rag_index.py ../../../ai-integration-course
# 139 chunks, 22,133 tokens embedded -> index.json
```

- Chunks are Markdown sections, split at headings.
- Each chunk starts with its file and heading: `[weeks/03/lab.md › 4. The second model]`.
- The whole course costs about **$0.0004** to embed.

Live: ask it

```
python3 rag_ask.py "When is the Week 3 lab due?"
```

- The four closest chunks go into the prompt, numbered.
- The system prompt: answer **only** from the excerpts, cite them, otherwise say *"I can't find that in the course documents."*
- The scores printed under the answer show why each chunk was chosen.

Five questions, with and without retrieval

#	Question	There to catch
1	When is the Week 3 lab due?	a fact in one place
2	Team Project 2’s share of the grade?	a number a model could guess
3	What if I submit late?	a policy across sections
4	Week 3’s second model?	a detail that changed
5	The campus parking policy?	not in the documents

> With retrieval: 5/5. Without: 1/5, and the one it "passed" was the refusal.

When RAG hurts: a stale page

```
Which Xiaomi MiMo model does this course use
as the fallback or second model?
```

- Retrieval finds **both** pages: Week 3's lab (`mimo-v2.6-flash`) and Week 1's setup (`mimo-v2.5`).
- The model answers `mimo-v2.5`, faithfully cited, and **out of date**.
- The pages disagree, and nothing in a chunk says which is newer.

> RAG is only as current as its documents. Add a current page and the answer changes.

Chain stage 3: the plan

Stage	Artifact	Written by	Approved by
Plan	`intent/*.md`	agent, after interviewing you	you
Design	`spec.md`	agent, from the intent	you, against the intent
Build	`plan.md`**, then code**	**agent**	**you, before any code**

- **Files** to create or change, and why.
- **Order** of work.
- **Risks:** what could break, the assumption behind each step, the blast radius.
- **Proof:** how you will know each step worked.

Interrogate the plan

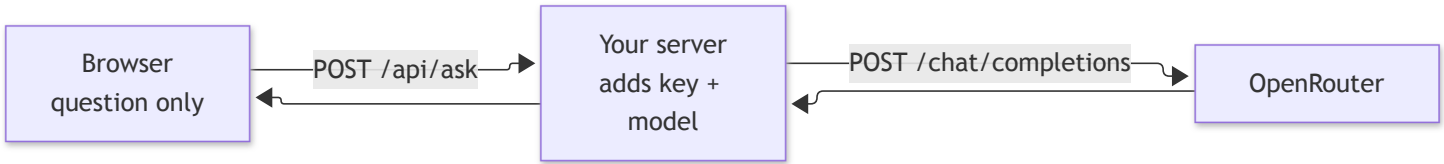
- "What could break in step 3, and how would I notice?"
- "How big is a chunk, and why that size?"
- "What does the proof step run, and what output means it worked?"
- "What happens when two documents disagree?"

> A plan you haven't questioned is the agent's plan, not yours.

Parallel work and blast radius

- **Subagents** work in their own context window: "summarize this repo," "find the auth code."
- **Git worktrees** give each parallel task its own checkout, so two agents don’t edit the same files.
- **Permissions** decide what an agent may do without asking. Wider permissions, bigger blast radius.
- Subagents inherit your model and cost tokens in parallel. Name a cheaper model for them.

The key stays on the server



- A key in a web page is a key anyone can copy: view source, network tab.
- Team Project 1 needs a server **even on your laptop**.
- `examples/key-on-server/` : the same page and contract in Python and Java. A web framework does the same job with less code.

Other ways to hold the key

Option	The key lives in
Your own server (today's example)	an environment variable on the server
Cloudflare Pages Functions / Workers	a Cloudflare secret
Vercel Functions	a Vercel environment variable
here.now proxy routes	a here.now service variable

- All have agent skills; the agent can deploy. **You** check the key isn't in the browser.
- Public means anyone can call it: add a password or rate limit, and a key with a credit limit.

Lab: ask the course

0 Template repo, `CLAUDE.md` lines

1 Intent and spec, briefly

2 **The plan**, questioned

3 Build from the plan; five questions

4 A stale page

5 Explain and save

weeks/04/lab.md · 45 minutes · Prof. Kousen circulates

Save evidence of the work

```
git add .
git commit -m "Ask-the-course bot with five-question eval"
git tag week04-submitted
git push -u origin main
git push origin week04-submitted
```

`plan.md` approved **before** the first code commit. `CHECKS.md`: with and without retrieval, and the stale page.

Before October 19

- **No class October 12** (Trinity Days).
- **October 19, 1:30:** the Week 4 submission; Team Project 1 `spec.md` and `plan.md`.
- Put the server in your TP1 plan.
- Week 5: vision models, and the Test stage.

Before you leave

- What does an embedding let you compare?
- Why tell the model to say "I can’t find that"?
- What should you ask a plan before you approve it?

Sources and next steps

- Ken Kousen, Retrieval-Augmented Generation (video) and kousen/ragdemo: LangChain, LangChain4j, Spring AI.
- OpenRouter embeddings · Artifact-chain template: `plan.md`.
- The AI-Native SDLC Playbook: the Build stage.
- Roy Fielding, *Architectural Styles and the Design of Network-based Software Architectures* (2000): REST.
- Reference RAG code and the server examples rehearsed September 28 on `minimax/minimax-m3` with `openai/text-embedding-3-small`.

Prepared September 28, 2026.